1 Training Procedure

Algorithm 1 LoRA-XS Training Procedure

Require: Pre-trained model f_{θ_0}, training data $\mathcal{D}$, target modules $\mathcal{M}$, rank r, learning rate η, epochs E

Ensure: Fine-tuned LoRA-XS parameters $\{R_j\}_{j \in \mathcal{M}}$

Freeze all pre-trained parameters θ_0.

for each target module $W_j \in \mathcal{M}$ **do**

Compute truncated singular value decomposition:

$$W_j = U_j \Sigma_j V_j^\top.$$

Keep the top-r singular components:

$$U_{j,r}, \quad \Sigma_{j,r}, \quad V_{j,r}.$$

Freeze $U_{j,r}$, $\Sigma_{j,r}$, and $V_{j,r}$.

Initialize the trainable matrix:

$$R_j \in \mathbb{R}^{r \times r}.$$

Define the LoRA-XS weight update:

$$\Delta W_j = U_{j,r} \Sigma_{j,r} R_j V_{j,r}^\top.$$

end for

for $e = 1$ to E **do**

for each mini-batch $(x, y) \in \mathcal{D}$ **do**

Compute prediction:

$$\hat{y} = f_{\theta_0, \{R_j\}}(x).$$

Compute training loss:

$$\mathcal{L} = \ell(\hat{y}, y).$$

Compute gradients with respect to trainable matrices:

$$\nabla_{R_j} \mathcal{L}, \quad j \in \mathcal{M}.$$

Update only LoRA-XS parameters:

$$R_j \leftarrow R_j - \eta \nabla_{R_j} \mathcal{L}.$$

end for

end for

return $\{R_j\}_{j \in \mathcal{M}}$